۸-۴ خلاصه

- نخ معرف واحد اساسی بهره‌وری پردازنده است و نخ‌های متعلق به پردازه‌ای یکسان، بسیاری از منابع پردازه، اعم از کد و داده‌ها را به‌اشتراک می‌گیرند.
- چهار نفع اصلی در کاربردهای چندنخی وجود دارد که عبارتند از: (۱) پاسخ‌گویی، (۲) اشتراک منبع، (۳) اقتصاد، و (۴) مقیاس‌پذیری.
- موقعی که نخ‌های متعددی پیشروی دارند همروندی وجود دارد، در حالی که موازی‌سازی موقعی وجود دارد که نخ‌های متعددی به‌صورت همزمان پیشروی دارند. در سیستمی با یک پردازنده‌ی منفرد، تنها همروندی امکان‌پذیر است؛ موازی‌سازی مستلزم یک سیستم چندهسته‌ای است که پردازنده‌های متعددی دارد.
- چندین چالش در طراحی کاربردهای چندنخی وجود دارد. آن‌ها عبارتند از تقسیم و موازنه‌ی کار، تقسیم داده‌ها میان نخ‌های متفاوت، و شناسایی هرگونه وابستگی داده‌ای. در نهایت، برنامه‌های چندنخی مخصوصاً در آزمون و عیب‌یابی چالش‌برانگیز هستند.
- موازی‌سازی داده‌ها زیرمجموعه‌های داده‌های یکسانی را در میان هسته‌های محاسباتی متفاوت توزیع کرده و عملیات یکسانی را بر روی هر هسته انجام می‌دهد. موازی‌سازی وظیفه داده‌ها را توزیع نمی‌کند بلکه وظایف را در میان هسته‌های متعدد توزیع می‌نماید. هر وظیفه یک عملیات یکتایی را اجرا می‌کند.
- برنامه‌های کاربر نخ‌های سطح کاربری ایجاد می‌کنند که بایستی در نهایت به‌منظور اجرا بر روی یک پردازنده به نخ‌های سطح هسته نگاشته شوند. مدل چند-به-یک نخ‌های سطح کاربری زیادی را به یک نخ هسته نگاشت می‌دهد. رویکردهای دیگر عبارتند از مدل‌های یک-به-یک و چند-به-چند.
- کتابخانه‌ی نخ یک API برای ایجاد و مدیریت نخ‌ها فراهم می‌کند. سه کتابخانه‌ی مرسوم نخ عبارتند از نخ‌سازی ویندوز، پوزیکس، و جاوا. ویندوز فقط برای سیستم ویندوز است، در حالی که نخ‌های پوزیکس برای سیستم‌های سازگار با پوزیکس همچون یونیکس، لینوکس، و macOS می‌باشند. نخ‌های جاوا روی هر سیستمی که از ماشین مجازی جاوا پشتیبانی نماید اجرا خواهند شد.
- نخ‌سازی ضمنی مشمول شناسایی وظایف—نه نخ‌ها—بوده و به زبان‌ها یا چارچوب‌های API امکان ایجاد و مدیریت نخ‌ها را می‌دهد. چندین رویکرد نخ‌سازی ضمنی وجود دارد، اعم از انبارهای نخ، چارچوب‌های fork-join، و اعزام مرکزی بزرگ. نخ‌سازی ضمنی در حال تبدیل به یک تکنیک مرسوم برای برنامه‌نویسان به‌منظور استفاده در برنامه‌نویسی کاربردهای همروند و موازی است.
- نخ‌ها ممکن است با استفاده از برکناری ناهمگام و یا تأخیردار خاتمه داده شوند. برکناری ناهمگام بلافاصله یک نخ را متوقف می‌کند، حتی اگر در میانه‌ی انجام یک به‌روزرسانی باشد. برکناری تأخیردار به نخ اطلاع می‌دهد که بایستی خاتمه یابد اما

به نخ اجازه می‌دهد تا با یک شیوه‌ی منظم خاتمه پیدا کند. در بسیاری از شرایط، برکناری تأخیردار به ختم ناهمگام ترجیح داده می‌شود.

- برعکس بسیاری از سیستم‌عامل‌های دیگر، لینوکس بین پردازه و نخ تفاوتی قایل نمی‌شود؛ در عوض، به هر یک از آن‌ها وظیفه اطلاق می‌کند. فراخوانی سیستمی clone() می‌تواند برای ایجاد وظایفی استفاده شود که بیشتر شبیه به پردازه‌ها و یا بیشتر شبیه به نخ‌ها رفتار می‌کنند.

خودآزمایی

۴-۱ سه مثال برنامه‌نویسی بیاورید که در آنها کارایی راه‌حل چندنخی از تک‌نخی بهتر است.

۴-۲ با استفاده از قانون Amdahl، تسریع حاصل یک کاربردی را که یک جزء موازی ۶۰ درصدی دارد برای (الف) دو هسته‌ی پردازشی و (ب) چهار هسته‌ی پردازشی محاسبه کنید.

۴-۳ آیا کارگزار وب چندنخی که در بخش ۴-۱ بیان شد، موازی‌سازی وظیفه یا داده‌ها را نمایش می‌دهد؟

۴-۴ دو تفاوت میان نخ‌های سطح کاربر و نخ‌های سطح هسته کدامند؟ تحت چه شرایطی، یک نوع از دیگری بهتر است؟

۴-۵ اعمالی را تشریح کنید که توسط هسته در برگردان‌متن میان نخ‌های سطح هسته انجام می‌شوند.

۴-۶ چه منابعی در زمان ایجاد یک نخ استفاده می‌شوند؟ چه تفاوتی در استفاده از آنها در زمان ایجاد یک پردازه وجود دارد؟

۴-۷ فرض کنید که یک سیستم‌عامل، نخ‌های سطح کاربر را با استفاده از مدل چند-به-چند به هسته می‌نگارد و این نگاشت از طریق LWP صورت می‌گیرد. به علاوه، سیستم به برنامه‌نویسان اجازه می‌دهد که نخ‌های بلادرنگ را برای استفاده در سیستم‌های بلادرنگ ایجاد نمایند. آیا پیوند یک نخ بلادرنگ به یک LWP ضرورت دارد؟ توضیح دهید.

تمرین‌ها

۴-۸ دو مثال برنامه‌نویسی ارائه دهید که در آن، کارایی راه‌حل چندنخی از تک‌نخی بهتر نیست.

۴-۹ تحت چه شرایطی یک راه‌حل چندنخی با استفاده از نخ‌های متعدد هسته، کارایی بهتری از راه‌حل تک‌نخی در یک سیستم تک‌پردازنده فراهم می‌کند؟

۴-۱۰ کدام یک از مؤلفه‌های حالت برنامه در زیر در میان نخ‌های یک پردازه‌ی چندنخی به اشتراک گذاشته می‌شوند؟

الف) مقادیر ثبات‌ها ب) حافظه‌ی heap

ج) متغیرهای سراسری د) حافظه‌ی پشته

۴-۱۱ آیا راه‌حل چندنخی با استفاده از چندین نخ سطح کاربر می‌تواند در یک سیستم چندپردازنده کارایی بهتری نسبت به یک سیستم تک‌پردازنده داشته باشد؟ توضیح دهید.

۴-۱۲ در فصل ۳ ، مرورگر کروم گوگل و عمل بازکردن هر tab جدید را در پردازه‌ای مجزا بحث کردیم. آیا همان منافع را می‌توان به جای طراحی کروم، با بازکردن هر tab جدید در یک نخ مجزا به‌دست آورد؟ توضیح دهید.

۴-۱۳ آیا داشتن همروندی، بدون موازی‌سازی ممکن است؟ توضیح دهید.

۴-۱۴ با استفاده از قانون Amdahl، تسریع حاصل را برای برنامه‌های زیر محاسبه کنید:

- ۴۰ درصد موازی با (الف) هشت هسته‌ی پردازشی و (ب) شانزده هسته‌ی پردازشی
- ۶۷ درصد موازی با (الف) دو هسته‌ی پردازشی و (ب) چهار هسته‌ی پردازشی
- ۹۰ درصد موازی با (الف) چهار هسته‌ی پردازشی و (ب) هشت هسته‌ی پردازشی

۴-۱۵ معین کنید که مسائل زیر، موازی‌سازی وظیفه یا داده‌ها را نمایش می‌دهند:

- با استفاده از یک نخ مجزا برای تولید یک thumbnail برای هر عکس در یک آلبوم
- ترانهاده‌کردن[1] یک ماتریس به‌صورت موازی
- یک کاربرد شبکه که در آن، یک نخ از شبکه خوانده و نخ دیگری در شبکه می‌نویسد
- برنامه‌ی حاصل‌جمع آرایه‌ی fork-join که در بخش ۴-۵-۲ مطرح شد
- سیستم اعزام مرکزی بزرگ

۴-۱۶ یک سیستم با دو پردازنده‌ی دوهسته‌ای، چهار پردازنده برای زمانبندی در دسترس دارد. یک برنامه‌ی پابند پردازنده در این سیستم در حال اجراست. تمامی ورودی در شروع برنامه انجام می‌شوند، موقعی که باید یک فایل منفرد باز شود. به طور مشابه، تمامی خروجی، درست قبل از اتمام برنامه انجام می‌شود، موقعی که نتایج برنامه بایستی در یک فایل منفرد نوشته شوند. برنامه مابین شروع و خاتمه، پابند پردازنده است. کار شما بهبود کارایی این برنامه با چندنخی کردن است. برنامه روی سیستمی اجرا می‌شود که از یک مدل نخ‌سازی یک-به-یک استفاده می‌کند (هر نخ کاربر به یک نخ هسته نگاشته می‌شود).

- چه تعداد نخ برای انجام ورودی و خروجی ایجاد خواهید کرد؟ توضیح دهید.

1. Transposing

- چه تعداد نخ برای بخش پابند پردازنده‌ی برنامه ایجاد خواهید کرد؟ توضیح دهید.

۴-۱۷ قطعه کد زیر را در نظر بگیرید:

```
pid_t pid;
pid = fork();
if (pid == 0) { /* child process */
        fork();
        thread_create(...);
}
fork();
```

الف) چه تعداد پردازه‌ی منحصر به فرد ایجاد می‌شوند؟

ب) چه تعداد نخ منحصر به فرد ایجاد می‌شوند؟

۴-۱۸ همان‌گونه که در بخش ۴-۷-۲ بیان شد، لینوکس میان پردازه‌ها و نخ‌ها تفاوتی قایل نمی‌شود. بلکه، لینوکس هر دو را یکسان در نظر گرفته و بسته به مجموعه‌ی پرچم‌هایی که با فراخوانی سیستمی clone() ارسال می‌شوند، وظیفه به یک پردازه یا به یک نخ می‌ماند. با این وجود، سیستم‌عامل‌های دیگری، همچون ویندوز، با پردازه‌ها و نخ‌ها به گونه‌ای متفاوت رفتار می‌کنند. چنین سیستم‌هایی نوعاً از یک نمادی استفاده می‌کنند که در آن، ساختمان‌داده‌ی یک پردازه، اشاره‌گرهایی به نخ‌های مجزای متعلق به پردازه دارد. این دو رویکرد را در مدل‌سازی پردازه‌ها و نخ‌های میان هسته مقایسه کنید.

۴-۱۹ برنامه‌ای که در شکل ۴-۲۳ نشان داده می‌شود از API نخ‌های پوزیکس استفاده می‌کند. خروجی برنامه در خطوط C و P، چه مقداری خواهد بود؟

```
#include <pthread.h>
#include <stdio.h>

int value = 0;
void *runner(void *param); /* the thread */
int main(int argc, char *argv[]){
        pid_t pid;
        pthread_t tid;
        pthread_attr_t attr;
        pid = fork();
        if (pid == 0) { /* child process */
                pthread_attr_init(&attr);
                pthread_create(&tid,&attr,runner,NULL);
                pthread_join(tid,NULL);
                printf("CHILD: value = %d",value); /* LINE C */
        }
        else if (pid > 0) { /* parent process */
                wait(NULL);
                printf("PARENT: value = %d",value); /* LINE P */
```

```
        }
}
void *runner(void *param) {
        value = 5;
        pthread_exit(0);
}
```

شکل ۴-۲۳ برنامه‌ی C تمرین ۴-۱۹.

۴-۲۰ یک سیستم چندهسته‌ای و یک برنامه‌ی چندنخی را که با استفاده از مدل نخ چند-به-چند نوشته شده است، در نظر بگیرید. فرض کنید که تعداد نخ‌های سطح کاربر در برنامه بیشتر از تعداد هسته‌های پردازشی سیستم است. در رابطه با عواقب کارایی شرایط زیر بحث کنید.

الف) تعداد نخ‌های هسته‌ی تخصیصی برنامه کمتر از تعداد هسته‌های پردازشی است.

ب) تعداد نخ‌های هسته‌ی تخصیصی برنامه برابر تعداد هسته‌های پردازشی است.

ج) تعداد نخ‌های هسته‌ی تخصیصی برنامه بیشتر از تعداد هسته‌های پردازشی بوده، اما از تعداد نخ‌های سطح کاربر کمتر است.

۴-۲۱ نخ‌های پوزیکس یک API برای مدیریت برکناری نخ فراهم می‌کنند. تابع pthread_setcancelstate() برای انتخاب حالت برکناری استفاده می‌شود. این تابع دارای prototype زیر است:

```
pthread_setcancelstate(int state, int *oldstate)
```

دو مقدار ممکن برای حالت، PTHREAD_CANCEL_ENABLE و PTHREAD_CANCEL_DISABLE می‌باشند.

با استفاده از قطعه کد نشان داده شده در شکل ۴-۲۴ دو نمونه از عملیات بیاورید که مناسبند مابین فراخوانی‌های غیرفعال‌کردن و فعال‌کردن برکناری نخ انجام شوند.

```
int oldstate;
pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, &oldstate);
/* What operations would be performed here? */
pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, &oldstate);
```

شکل ۴-۲۴ برنامه‌ی C تمرین ۴-۲۱.

مسائل برنامه‌نویسی

۴-۲۲ یک برنامه‌ی چندنخی بنویسید که مقادیر آماری مختلف لیستی از اعداد را محاسبه

می‌کند. به این برنامه یک دنباله از اعداد از طریق خط فرمان ارسال شده و سپس سه نخ کارگر مجزا ایجاد خواهد کرد. یک نخ، میانگین اعداد، دومی مقدار بیشینه و سومی مقدار کمینه را تعیین خواهد کرد. برای مثال، فرض کنید که اعداد صحیح زیر به برنامه‌ی شما ارسال می‌شوند:

```
90 81 78 95 79 72 85
```

برنامه خروجی زیر را خواهد داد:

```
The average value is 82
The minimum value is 72
The maximum value is 95
```

متغیرهای نمایش‌دهنده‌ی مقادیر میانگین، کمینه و بیشینه به صورت سراسری ذخیره خواهند شد. این مقادیر را نخ‌های کارگر مشخص خواهند کرد و نخ والد، مقادیر را به محض خروج کارگرها خروجی خواهد نمود. (تعمیم واضح این برنامه آن است که نخ‌های دیگری ایجاد شوند تا مقادیر آماری دیگری، همچون میانه و انحراف معیار را تعیین کنند.)

۴-۲۳ یک برنامه‌ی چندنخی بنویسید که اعداد اول را نمایش می‌دهد. این برنامه بایستی به این صورت کار کند: کاربر برنامه را اجرا کرده و یک عدد از خط فرمان وارد خواهد کرد. در ادامه، برنامه یک نخ مجزایی ایجاد می‌کند که تمامی اعداد اول کوچک‌تر یا مساوی با عدد وارد شده توسط کاربر را نمایش می‌دهد.

۴-۲۴ یک روش جالب محاسبه‌ی π استفاده از تکنیکی معروف به ***مونته کارلو***[1] است که مشمول فرایند تصادفی است. این تکنیک به این صورت کار می‌کند: فرض کنید دایره‌ای دارید که در داخل یک مربع، آن‌گونه که در شکل ۴-۲۵ نشان داده می‌شود، محاط شده است. (شعاع این دایره را ۱ فرض کنید.)

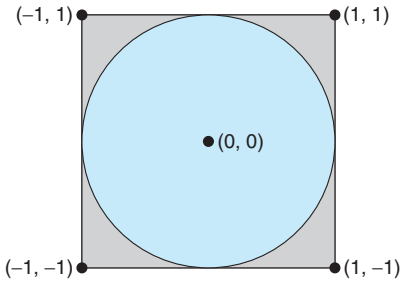
- اولاً، یک سری نقاط تصادفی به صورت مختصات (x, y) تولید کنید. این نقاط بایستی در میان مختصات دکارتی محدود به مربع قرار بگیرند. از کل تعداد نقاط تصادفی که تولید می‌شوند، برخی در داخل دایره واقع خواهند شد.
- سپس، π را با انجام محاسبه‌ی زیر تقریب بزنید:

$$\pi = 4 \times (\text{تعداد نقاط درون دایره}) / (\text{تعداد کل نقاط})$$

یک نسخه‌ی چندنخی از این الگوریتم بنویسید که نخ مجزایی برای تولید نقاط تصادفی ایجاد می‌کند. نخ، تعداد نقاطی را که در داخل دایره واقع می‌شوند شمارش کرده و نتیجه را در یک متغیر سراسری ذخیره می‌کند. موقعی که این نخ خارج می‌شود، نخ والد مقدار تقریبی π را محاسبه و خروجی خواهد نمود. آزمایش و تجربه‌ی تعداد نقاط تصادفی تولید شده ارزشمند است. به عنوان یک

1. Monte Carlo

قاعده‌ی کلی، هر چه تعداد نقاط بیشتر شود، تقریب به‌دست آمده به π نزدیک‌تر می‌شود.



شکل ۴-۲۵ تکنیک مونته کارلو برای محاسبه‌ی π.

در مجموعه‌ی برنامه‌های دانلود شده برای این کتاب، یک برنامه‌ی ساده پیدا خواهید کرد که تکنیکی برای تولید اعداد تصادفی، در کنار تعیین این که نقطه‌ی تصادفی (x, y) در دایره واقع می‌شود یا خیر، ارائه می‌کند.

این تمرین را در فصل ۶، با استفاده از مطالب مرتبطی از آن فصل تغییر می‌دهیم.

۴-۲۵ تمرین ۴-۲۲ را تکرار کنید، اما به جای استفاده از یک نخ مجزا برای تولید نقاط تصادفی، برای موازی‌سازی تولید نقاط از OpenMP استفاده کنید. دقت کنید که محاسبه‌ی π را در ناحیه‌ی موازی نگذارید، چرا که تنها یک بار π را محاسبه خواهید کرد.

۴-۲۶ کارگزار تاریخ مبتنی بر سوکت (شکل ۳-۲۷) فصل ۳ را چنان اصلاح کنید که، کارگزار هر یک از درخواست‌های کارفرما را با نخ مجزایی سرویس بدهد.

۴-۲۷ دنباله‌ی فیبوناتچی، سری اعداد ۰، ۱، ۱، ۲، ۳، ۵، ۸، ... است. این سری را می‌توان به صورت فرمول زیر بیان کرد:

$$fib_0 = 0$$
$$fib_1 = 1$$
$$fib_n = fib_{n-1} + fib_{n-2}$$

یک برنامه‌ی چندنخی بنویسید که دنباله‌ی فیبوناتچی را تولید کند. این برنامه بایستی به این صورت کار کند: کاربر از طریق خط فرمان، تعداد جملات دنباله‌ی فیبوناتچی را که برنامه تولید خواهد کرد، وارد می‌کند. سپس، برنامه نخ مجزایی ایجاد می‌کند که جملات فیبوناتچی را با قراردادن دنباله در ساختمان‌داده‌هایی که می‌تواند در میان نخ‌ها مشترک باشد (شاید برای این کار، آرایه ساختمان‌داده‌ی مناسبی باشد)، تولید خواهد نمود. زمانی که نخ اجرای خود را به پایان می‌رساند،

نخ والد دنباله‌ای را که نخ فرزند تولیدکرده نمایش خواهد داد. چون نخ والد نمی‌تواند تا زمانی‌که نخ فرزند خاتمه نیافته است، دنباله‌ی فیبوناتچی را نمایش بدهد، نخ والد به ناچار بایستی با استفاده از تکنیک‌های مطرح شده در بخش ۴-۳، به انتظار اتمام کار نخ فرزند بماند.

۴-۲۸ مسأله‌ی برنامه‌نویسی تمرین ۳-۲۰ فصل ۳ را اصلاح کنید که از شما می‌خواست یک مدیر pid طراحی کنید. این تغییر شامل نوشتن یک برنامه‌ی چندنخی است که جواب شما به تمرین ۳-۲۰ را می‌آزماید. شما تعدادی نخ ایجاد خواهید کرد—برای مثال، ۱۰۰—و هر نخ یک pid درخواست خواهد داد، برای یک دوره‌ی زمانی تصادفی خوابیده، و در ادامه، pid را رها می‌کند. (خوابیدن یک دوره‌ی زمانی تصادفی تقریب یک استفاده‌ی نوعی از pid است که در آن، یک pid به یک پردازه‌ی جدیدی گمارده می‌شود، پردازه اجرا شده و سپس خاتمه می‌یابد، و pid با ختم پردازه رها می‌گردد.) در سیستم‌های یونیکس و لینوکس، خواب‌سپاری با تابع sleep() انجام می‌شود که یک مقدار عددی صحیح، معرف تعداد ثانیه‌های خواب به آن ارسال می‌شود. این مسئله در فصل ۷ تغییر خواهد یافت.

۴-۲۹ تمرین ۳-۲۵ فصل ۳، به طراحی یک کارگزار انعکاس با استفاده از API نخ جاوا مربوط است. این کارگزار تک‌نخی است، به این معنی که کارگزار نمی‌تواند به کارفرماهای اکو همروند پاسخ بدهد مادامی که کار کارفرمای فعلی تمام نشده است. راه‌حل تمرین ۳-۲۵ را چنان اصلاح کنید که کارگزار اکو، درخواست هر یک از کارفرماها را با نخ مجزایی سرویس بدهد.

پروژه‌های برنامه‌نویسی

پروژه‌ی ۱—اعتبارسنج جواب سودوکو

معمای سودوکو[1] از یک شبکه‌ی ۹ × ۹ استفاده می‌کند که در آن هر ستون و سطر، همین‌طور هر نه زیرشبکه‌ی ۳ × ۳ آن، بایستی تمامی ارقام ۱ تا ۹ را در بر بگیرند. شکل ۴-۲۶، نمونه‌ای از یک معمای معتبر سودوکو را نشان می‌دهد. این پروژه شامل طراحی یک برنامه‌ی چندنخی است که تعیین می‌کند آیا جواب یک معمای سودوکو معتبر است یا خیر.

چندین روش چندنخی متفاوت برای این برنامه وجود دارند. یک راهبرد پیشنهادی این است که نخ‌هایی ایجاد شوند که معیارهای زیر را بررسی می‌کنند:

- یک نخ بررسی می‌کند که هر ستون شامل ارقام ۱ تا ۹ است.
- یک نخ بررسی می‌کند که هر سطر شامل ارقام ۱ تا ۹ است.

1. Sudoku

- نه نخ که بررسی می‌کنند زیرشبکه‌های ۳ × ۳ شامل ارقام ۱ تا ۹ هستند.

حاصل این کار، در مجموع به یازده نخ مجزا برای اعتبارسنجی یک معمای سودوکو می‌انجامد. با این وجود، بهتر است که شما برای این پروژه نخ‌های بیشتری هم ایجاد کنید. برای مثال، به جای ایجاد یک نخ که تمامی نه ستون را بررسی می‌کند، می‌توانید نه نخ مجزا ایجاد کنید که هر یک از آنها یک ستون را بررسی می‌کنند.

6	2	4	5	3	9	1	8	7
5	1	9	7	2	8	6	3	4
8	3	7	6	1	4	2	9	5
1	4	3	8	6	5	7	2	9
9	5	8	2	4	7	3	6	1
7	6	2	3	9	1	4	5	8
3	7	1	9	5	6	8	4	2
4	9	6	1	8	2	5	7	3
2	8	5	4	7	3	9	1	6

شکل ۴-۲۶ جوابی برای یک معمای سودوکوبا ابعاد ۹ × ۹.

۱—ارسال پارامترها به هر یک از نخ‌ها

نخ والد، نخ‌های کارگر را با ارسال مکانی که بایستی در شبکه‌ی سودوکو بررسی کنند، ایجاد خواهد کرد. در این مرحله لازم است پارامترهای متعددی به هر یک از نخ‌ها ارسال شود. ساده‌ترین رویکرد، ایجاد یک ساختمان‌داده با استفاده از struct است. برای مثال، رکورد سطر و ستون ارسالی که بایستی نخ اعتبارسنجی را شروع کند به صورت زیر خواهد بود:

```
/* structure for passing data to threads */
typedef struct
{
        int row;
        int column;
} parameters;
```

هر دوی برنامه‌های نخ‌های پوزیکس و ویندوز، نخ‌های کارگر را با استفاده از یک راهبرد مشابه آنچه که در زیر نشان داده می‌شود ایجاد خواهند کرد:

```
parameters *data = (parameters *) malloc(sizeof(parameters));
data->row = 1;
data->column = 1;
/* Now create the thread passing it data as a parameter */
```

اشاره‌گر data به تابع pthread_create() (در نخ‌های پوزیکس) و یا تابع CreateThread() (در ویندوز) ارسال خواهد شد، که آن هم به نوبت خود، آن را به عنوان پارامتری به تابعی ارسال

خواهد کرد که به عنوان یک نخ مجزا اجرا می‌شود.

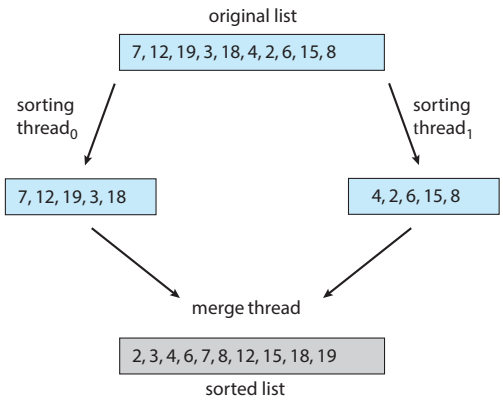
۲—برگرداندن نتایج به نخ والد

هر کارگر به وظیفه‌ی تعیین اعتبار یک ناحیه‌ی خاص معمای سودوکو گمارده می‌شود. همین که کارگری این بررسی را انجام داد، بایستی نتایج آن را به والد ارسال کند. یک روش خوب انجام این کار، ایجاد یک آرایه از مقادیر صحیح است که برای تک‌تک نخ‌ها قابل رؤیت است. ایندکس iام این آرایه متناظر نخ کارگر iام است. اگر کارگری مقدار متناظر خود را ۱ انتخاب کند، نشان می‌دهد که ناحیه‌ی معمای سودوکو برای آن معتبر است. مقدار ۰ عکس این را نشان می‌دهد. موقعی که تمامی نخ‌های کارگر تکمیل شدند، نخ والد، هر یک از واردهای آرایه را به منظور تعیین این که معمای سودوکو معتبر است یا خیر، بررسی می‌کند.

پروژه‌ی ۲—برنامه‌ی مرتب‌سازی چندنخی

یک برنامه‌ی مرتب‌سازی چندنخی بنویسید که به این صورت کار می‌کند: لیستی از اعداد صحیح به دو لیست کوچک‌تر با اندازه‌ی یکسان تقسیم می‌شود. دو نخ مجزا (که به آنها اصطلاحاً ***نخ‌های مرتب‌سازی*** می‌گوییم) هر یک از زیرلیست‌ها را با یک الگوریتم مرتب‌سازی به انتخاب شما مرتب می‌کنند. سپس، دو زیرلیست با نخ سومی—**نخ ادغام‌کننده**—که دو زیرلیست را در یک لیست مرتب منفرد ادغام می‌کند، ادغام می‌شوند.

چون داده‌های سراسری مابین تمامی نخ‌ها مشترک هستند، شاید ساده‌ترین راه برپایی داده‌ها ایجاد یک آرایه‌ی سراسری است. هر یک از نخ‌های مرتب‌ساز بر روی یک نیمه از این آرایه کار خواهند کرد. آرایه‌ی سراسری دومی با همان اندازه‌ی آرایه‌ی اعداد صحیح نامرتب نیز بنا خواهد شد. سپس، نخ ادغام‌کننده دو زیرلیست را در این آرایه‌ی دوم ادغام خواهد کرد. این برنامه به صورت تصویری، مثل شکل ۴–۲۷ ساختار می‌یابد.



شکل ۴–۲۷ مرتب‌سازی چندنخی.

این پروژه‌ی برنامه‌نویسی لازم دارد که پارامترهایی به نخ‌های مرتب‌سازی ارسال شوند. به طور خاص، لازم است که ایندکس شروع مرتب‌سازی برای هر یک از نخ‌ها مشخص شود. برای جزئیات ارسال پارامترها به یک نخ، به دستورالعمل‌های پروژه‌ی ۱ رجوع کنید.

نخ والد به محض خروج تمامی نخ‌های مرتب‌سازی، آرایه‌ی مرتب‌شده را خروجی خواهد نمود.

پروژه‌ی ۳—برنامه‌ی مرتب‌سازی Fork-Join

پروژه‌ی قبل (برنامه‌ی مرتب‌سازی چندنخی) را با استفاده از API موازی‌سازی fork-join جاوا پیاده‌سازی کنید. این پروژه در دو نسخه توسعه داده خواهد شد. هر نسخه یک الگوریتم مرتب‌سازی تقسیم و غلبه‌ی متفاوتی را پیاده‌سازی خواهد کرد:

1- Quicksort
2- Mergesort

پیاده‌سازی Quicksort از الگوریتم Quicksort برای تقسیم لیست عناصری که قرار است مرتب شوند به *نیمه‌ی* چپ و *نیمه‌ی راست*، بر اساس مکان مقدار محوری[1] استفاده خواهد کرد. الگوریتم Mergesort لیست را به دو نیمه‌ی تقریباً مساوی تقسیم خواهد کرد. در هر دو الگوریتم Quicksort و Mergesort، موقعی که اندازه‌ی لیست مرتب‌سازی کمتر از یک مقدار آستانه می‌افتد (برای مثال، اندازه‌ی لیست برابر ۱۰۰ یا کمتر می‌شود)، مستقیماً یک الگوریتم ساده همچون مرتب‌سازی انتخابی یا درجی را به‌کار می‌برند. اکثر کتاب‌های ساختمان‌داده‌ها این دو الگوریتم مرتب‌سازی تقسیم و غلبه‌ی معروف را توضیح می‌دهند.

کلاس SumTask نشان داده شده در بخش ۴-۵-۲-۱ کلاس RecursiveTask را گسترش می‌دهد که نتیجه کلاس ForkJoinTask می‌شود. چون این تمرین مشمول مرتب‌سازی آرایه‌ای است که به وظیفه ارسال می‌شود، اما هیچ مقداری برگردانده نمی‌شود، شما در عوض یک کلاسی ایجاد خواهید کرد که RecursiveAction را گسترش می‌دهد، یک ForkJoinTask که نتیجه‌ای به‌بار نمی‌آورد (شکل ۴-۱۹ را ببینید).

اشیائی که به هر یک از الگوریتم‌های مرتب‌سازی ارسال می‌شوند لازم است که رابط Comparable جاوا را پیاده‌سازی کنند و این کار نیاز است که در تعریف کلاس هر یک از الگوریتم‌های مرتب‌سازی منعکس شود. کد اصلی دانلود شده‌ی این کتاب شامل کد جاواست که مبانی شروع این پروژه را فراهم می‌کند.

1. Pivot